

Lab 3-4: Research Project Proposal

Visualization-Oriented Logs for Debugging Distributed Systems

Bujor Mihai Alexandru
bujormihaialexandru@gmail.com
Babeş-Bolyai University

Paper Outline

Abstract

This research proposes a visualization-oriented logging approach for debugging cloud-native distributed systems. We design a log format incorporating causality metadata (vector clocks, span relationships) and develop an end-to-end prototype for Kubernetes environments. Through controlled experiments, we evaluate debugging effectiveness, performance overhead, and usability compared to traditional logging and distributed tracing systems.

1. Introduction

- Motivation: complexity of debugging microservices architectures
- Problem: lack of causality visibility in distributed executions
- Solution: logs designed explicitly for visualization and debugging
- Contributions: log format, prototype, empirical evaluation, design principles

2. Background and Related Work

2.1 Distributed Systems Fundamentals

Asynchronous communication, partial failures, time and causality (Lamport timestamps [1], vector clocks [2])

2.2 Debugging Approaches

- Traditional logging (text-based, grep analysis)
- Distributed tracing: Dapper [3], Jaeger, Zipkin
- Visualization tools: ShiViz [4], ViSiDiA [5]
- Debuggers: MaceODB [6]

2.3 Observability in Cloud-Native Systems

- OpenTelemetry standard
- Service mesh observability (Istio, Linkerd)
- Performance analysis tools [7], [8], [9]

2.4 Gap Analysis

Existing tools separate tracing from visualization, lack rigorous causality tracking, and have high production overhead.

3. Problem Formulation

3.1 Research Questions

RQ1: What minimal metadata in logs enables effective distributed debugging?

RQ2: How much does interactive visualization reduce debugging time versus traditional approaches?

RQ3: What is the performance overhead of visualization-oriented logging?

RQ4: How easily can developers adopt this approach?

4. Proposed Approach

4.1 Log Format Design

- Mandatory fields: timestamp, service_id, trace_id, span_id, event_type
- Causality metadata: vector_clock, parent_span_id
- Optional: request/response payloads, state snapshots

4.2 System Architecture

- **Instrumentation:** Library with automatic ID generation, context propagation
- **Collection:** Agents forwarding to message queue
- **Processing:** Stream processing for event correlation
- **Storage:** Time-series database
- **Visualization:** Interactive web-based UI

4.3 Key Capabilities

Timeline view, service dependency graph, request flow tracing, causality inspection, state reconstruction

5. Implementation

Prototype includes instrumentation library, collection pipeline, stream processing, storage backend, and interactive visualization interface with filtering and zooming capabilities.

6. Evaluation

6.1 Experimental Design

Experiment 1: Debugging Effectiveness

Within-subjects study (N=12-15 engineers) debugging 3 scenarios with traditional logging, Jaeger, and proposed system. Measure time to diagnosis and success rate.

Experiment 2: Performance Overhead

Benchmark microservices application under varying load (100-5000 RPS) and sampling rates (1%, 10%, 100%). Compare latency, throughput, resource usage against baselines.

Experiment 3: Scalability

Test with 5-50 services at varying loads. Measure log volume, storage requirements, query latency.

Experiment 4: Usability

User study (N=10) with System Usability Scale questionnaire and qualitative interviews.

6.2 Metrics

- Debugging: Mean Time to Diagnosis (MTTD), success rate
- Performance: p50/p95/p99 latency, throughput, CPU/memory usage
- Scalability: events/sec, storage GB/hour, query latency
- Usability: SUS score, qualitative feedback

6.3 Expected Results

- 30-50% MTTD reduction for complex failures
- < 5% latency overhead at 10% sampling
- Linear scaling to 10K RPS
- SUS score >70

7. Discussion

Analysis of results, comparison with baselines, limitations (scalability constraints, generalizability), threats to validity.

8. Contributions

C1: Visualization-oriented log format specification with causality metadata

C2: End-to-end prototype for Kubernetes with native integration

C3: Empirical evaluation of debugging effectiveness and overhead

C4: Design principles for debuggable distributed systems

Bibliography

Bibliography

- [1] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, July 1978, doi: 10.1145/359545.359563.
- [2] F. Mattern, “Virtual Time and Global States of Distributed Systems,” in *Proceedings of the Workshop on Parallel and Distributed Algorithms*, North-Holland, 1989, pp. 215–226.
- [3] B. H. Sigelman *et al.*, “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,” Technical Report, 2010.
- [4] I. Beschastnikh, Y. Xu, M. Goel, and B. Mehta, “ShiViz: A Platform for Visualizing Distributed Executions,” *ACM Transactions on Software Engineering and Methodology*, vol. 29, no. 2, pp. 1–42, 2020, doi: 10.1145/3375633.
- [5] J. Chalopin, Y. Métivier, and T. Morsellino, “ViSiDiA: A Platform for the Visualization and Simulation of Distributed Algorithms,” *Electronic Notes in Theoretical Computer Science*, vol. 268, pp. 45–53, Dec. 2011, doi: 10.1016/j.entcs.2010.12.015.
- [6] T. Dao, J. Shneidman, B. Ports, and M. Seltzer, “MaceODB: A Debugging Tool for Distributed Systems,” in *Proceedings of the 1st USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2009.
- [7] R. R. Sambasivan *et al.*, “Diagnosing Performance Changes by Comparing Request Flows,” in *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, Boston, MA, 2011, pp. 43–56.
- [8] X. Zhao *et al.*, “lprof: A Non-intrusive Request Flow Profiler for Distributed Systems,” in *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Broomfield, CO, 2014, pp. 629–644.
- [9] M. Chow, D. Meisner, J. Flinn, D. Peek, and T. F. Wenisch, “The Mystery Machine: End-to-end Performance Analysis of Large-scale Internet Services,” in *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Broomfield, CO, 2014, pp. 217–231.

Research Methodology

Hypothesis

Structured logs with visualization-specific metadata (trace IDs, vector clocks, service dependencies) reduce mean time to diagnosis in cloud-native systems by 30-50% versus traditional logging, with acceptable overhead (< 5% latency at 10% sampling).

Methodology

Phase 1: Analysis (2 weeks)

- Literature review and tool analysis (Jaeger, Zipkin, ShiViz, OpenTelemetry)
- Gap identification
- Requirements specification

Deliverable: Comparative analysis document

Phase 2: Design (3 weeks)

- Log format specification (JSON schema)
- Architecture design (collection, processing, storage, visualization)
- UI mockups (timeline, dependency graph, flow diagram)
- Evaluation protocol design

Deliverable: Design specification with diagrams and schemas

Phase 3: Implementation (6 weeks)

- Week 1-2: Instrumentation library with vector clocks, async logging
- Week 3: Collection agents and stream processing
- Week 4-5: Visualization frontend with interactive components
- Week 6: Container orchestration integration and testing

Deliverable: Functional prototype with documentation

Phase 4: Evaluation (4 weeks)

- Week 1: Deploy test application (Google Online Boutique), inject bugs
- Week 2-3: Run experiments (debugging study, performance benchmarks, scalability tests)
- Week 4: User study and data analysis

Deliverable: Experimental results and statistical analysis

Phase 5: Writing (2 weeks)

Complete paper with results, discussion, conclusions

Original Approach

This work differs from existing approaches through:

1. **Debugging-first design:** Optimized for debugging workflows, not general observability
2. **Integrated causality:** Vector clocks in log format for rigorous happens-before tracking
3. **End-to-end integration:** Unified collection-to-visualization pipeline (unlike Jaeger + separate viz tools)
4. **Cloud-native optimization:** Native Kubernetes support with service mesh integration
5. **Adaptive overhead:** Higher sampling for errors, lower for successful requests

Experiments

Debugging Time Comparison

12-15 engineers debug 3 failure scenarios (cascade failure, race condition, bottleneck) using traditional logging, Jaeger, and proposed system. Counterbalanced within-subjects design. Measure MTTD, success rate, tool interactions.

Analysis: Repeated measures ANOVA, pairwise comparisons

Performance Overhead

Factorial benchmark: 4 logging approaches × 3 sampling rates × 3 load levels. 10-minute runs with 3 replications. Measure latency, throughput, resource usage.

Analysis: Factorial ANOVA, overhead percentages relative to baseline

Scalability Assessment

Stress test with 5-50 services at 100-10K RPS. 1-hour runs. Measure log volume, storage growth, processing lag, query latency.

Analysis: Regression modeling of scaling relationships

Usability Study

10 engineers complete debugging tasks, SUS questionnaire, semi-structured interviews.

Analysis: Descriptive statistics, thematic analysis of interviews

Original Contribution

Research Contributions

Contribution 1: Log Format Specification

First structured format explicitly incorporating vector clocks and debugging-specific metadata. Provides standard for reproducible research.

Contribution 2: Prototype System

End-to-end implementation demonstrating feasibility, released open-source for adoption and extension.

Contribution 3: Empirical Evaluation

Systematic comparison of debugging approaches with controlled experiments on realistic microservices.

Contribution 4: Design Principles

Synthesized insights: (1) capture causality not just timestamps, (2) design for interactive exploration, (3) minimize instrumentation burden, (4) adaptive overhead management, (5) domain-specific optimization.

Differentiation from Related Work

vs. Distributed Tracing (Jaeger/Zipkin):

Adds rigorous causality (vector clocks vs. timestamps), debugging-oriented visualization (interactive state inspection vs. basic timeline), debugging-specific metadata.

vs. ShiViz:

Production-ready integration (vs. standalone tool), cloud-native optimization (vs. generic), real-time collection (vs. manual upload), scalability for large systems.

vs. OpenTelemetry:

Specialized debugging format (vs. generic observability), mandated causality metadata (vs. optional), integrated visualization tooling (vs. specification only).

Impact

Academic: Publications at OSDI/NSDI/ICSE, open artifacts for reproducibility, foundation for automated debugging research

Industry: Practical tool for microservices debugging, integration potential with Kubernetes/Istio, guidance for observability platforms

Broader: Reduced service downtime through faster debugging, improved developer productivity, more reliable systems